

Dynamic Programming

1 Introduction to Dynamic Programming

- weighted interval scheduling
- the design of a recursive solution
- memoizing the recursion
- longest common subsequence

2 Principles of Dynamic Programming

- another algorithm design technique
- memoization or iteration over subproblems
- the direct iterative algorithm
- a basic outline of dynamic programming

CS 401/MCS 401 Lecture 12
Computer Algorithms I
Jan Verschelde, 15 July 2026

Dynamic Programming

1 Introduction to Dynamic Programming

- **weighted interval scheduling**
- the design of a recursive solution
- memoizing the recursion
- longest common subsequence

2 Principles of Dynamic Programming

- another algorithm design technique
- memoization or iteration over subproblems
- the direct iterative algorithm
- a basic outline of dynamic programming

weighted interval scheduling

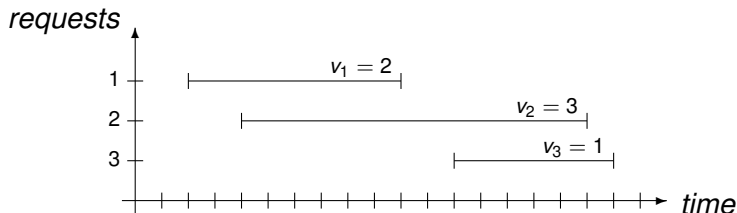
Input: n requests (s_i, f_i, v_i) , $i = 1, 2, \dots, n$,

s_i is the start time of request i , f_i is the finish time of request i ,

v_i is the value of request i .

The requests are sorted: $f_1 \leq f_2 \leq \dots \leq f_n$.

For example, $n = 3$:



Goal: Select a set $S \subseteq \{1, 2, \dots, n\}$ of mutually compatible intervals, to maximize the sum of values $\sum_{i \in S} v_i$.

interval scheduling

our first greedy algorithm

Our problem is to schedule requests for a single resource.

Input: n requests $\{ 1, 2, \dots, n \}$,
each request is a time interval $[s(i), f(i)]$, where
 $s(i)$ is the starting time of the i -th request; and
 $f(i)$ is the finishing time of the i -th request, $f(i) > s(i)$.

Output: a subset S of $\{ 1, 2, \dots, n \}$ so that
no two intervals in S overlap; and
 $\#S$ is as large as possible.

The goal is to schedule as many requests as possible.

Solution: Schedule the Request that Finishes First.

scheduling requests with deadlines

another greedy algorithm

We have one single resource and n requests.

Input: $\{ 1, 2, \dots, n \}$, a set of n requests,
request i has a deadline d_i
and needs time t_i to complete, for $i = 1, 2, \dots, n$.

We want to schedule all requests so that

- there are no overlaps in the time intervals;
- requests should meet their deadlines (or finish not too late).

Solution: the Earliest Deadline First Algorithm.

weighted interval scheduling

Input: n requests (s_i, f_i, v_i) , $i = 1, 2, \dots, n$,
 s_i is the start time of request i , f_i is the finish time of request i ,
 v_i is the value of request i .

The requests are sorted: $f_1 \leq f_2 \leq \dots \leq f_n$.

Goal: Select a set $S \subseteq \{1, 2, \dots, n\}$ of mutually compatible intervals,
to maximize the sum of values $\sum_{i \in S} v_i$.

As the other scheduling problems, we have an optimization problem.

However, there is no greedy algorithm that works.

We introduce another algorithm design technique.

Dynamic Programming

1 Introduction to Dynamic Programming

- weighted interval scheduling
- **the design of a recursive solution**
- memoizing the recursion
- longest common subsequence

2 Principles of Dynamic Programming

- another algorithm design technique
- memoization or iteration over subproblems
- the direct iterative algorithm
- a basic outline of dynamic programming

the leftmost interval that ends before j begins

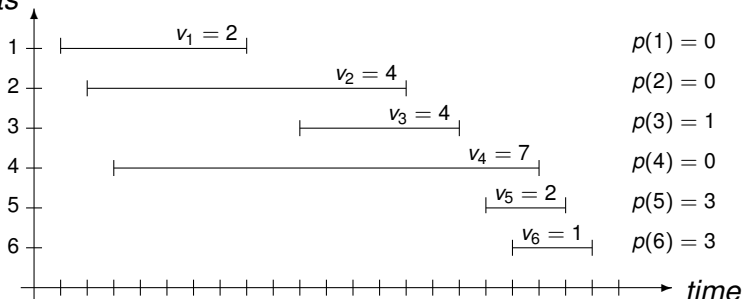
Definition (1)

For n requests (s_i, f_i, v_i) , $i = 1, 2, \dots, n$, *the function $p(j)$* for request j is

- $p(j) = 0$ is no request $i < j$ is disjoint from j , else
- $p(j)$ is the largest index $i < j$ such that i and j are disjoint.

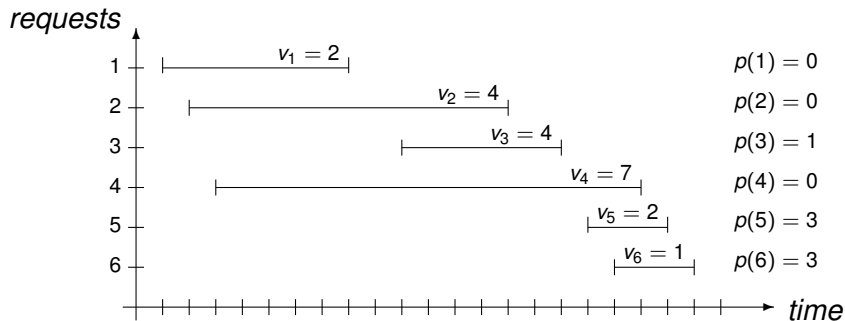
For example, $n = 6$:

requests



exploring a dichotomy

Either the last interval belongs to an optimal solution, or it does not.



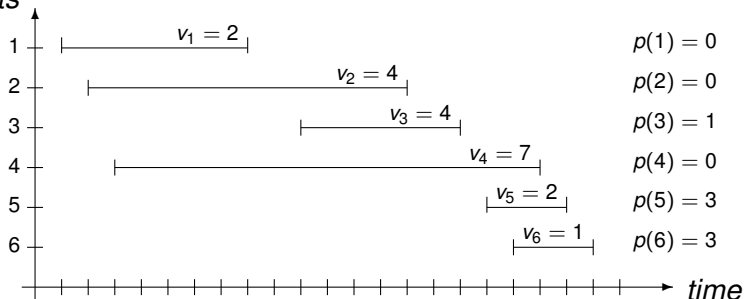
If the last interval n belongs to an optimal solution $\mathcal{O}$, then

- 1 All intervals $p(n) + 1, p(n) + 2, \dots$, all overlap with n .
- 2 $\mathcal{O}$ must contain an optimal solution to the problem defined by $\{ 1, 2, \dots, p(n) \}$.

exploring a dichotomy

Either the last interval belongs to an optimal solution, or it does not.

requests



If the last interval n does not belong to an optimal solution $\mathcal{O}$, then $\mathcal{O}$ is the optimal solution to the problem defined by $\{ 1, 2, \dots, n-1 \}$.

on optimal solutions of subproblems

For any request $j \in \{1, 2, \dots, n\}$,
denote $\mathcal{O}_j$ the optimal solution for the requests $\{1, 2, \dots, j\}$.
Let $\text{OPT}(j)$ be the value of the optimal solution $\mathcal{O}_j$.

For n requests (s_i, f_i, v_i) , $i = 1, 2, \dots, n$,
recall the function $p(j)$ for request j is

- $p(j) = 0$ if no request $i < j$ is disjoint from j , else
- $p(j)$ is the largest index $i < j$ such that i and j are disjoint.

Lemma (2)

$$\text{OPT}(j) = \max(v_j + \text{OPT}(p(j)), \text{OPT}(j-1)).$$

Either $j \in \mathcal{O}_j$ or $j \notin \mathcal{O}_j$:

- If $j \in \mathcal{O}_j$, then $\text{OPT}(j) = v_j + \text{OPT}(p(j))$.
- If $j \notin \mathcal{O}_j$, then $\text{OPT}(j) = \text{OPT}(j-1)$.

accept request j or not?

How to decide whether request j belongs to an optimal solution?

Lemma (3)

Request j belongs to an optimal solution of $\{1, 2, \dots, j\}$ if and only if $v_j + \text{OPT}(p(j)) \geq \text{OPT}(j - 1)$.

The lemma leads to a recurrence relation that expresses the optimal solution (or its value) in terms of the optimal solutions of smaller subproblems.

a recursive algorithm

Compute-Opt(j, v, p)

Input: j , the index of the current request,
 v , the array of values for each request,
 p , the function p for the requests.

if $j = 0$ then

return 0

else

return max(Compute-Opt($p(j), v, p$) + v_j ,
Compute-Opt($j - 1, v, p$))

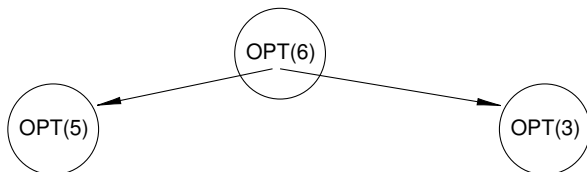
the root of the tree of subproblems

$$\text{OPT}(n) = \max(\text{OPT}(n-1), v_n + \text{OPT}(p(n)))$$

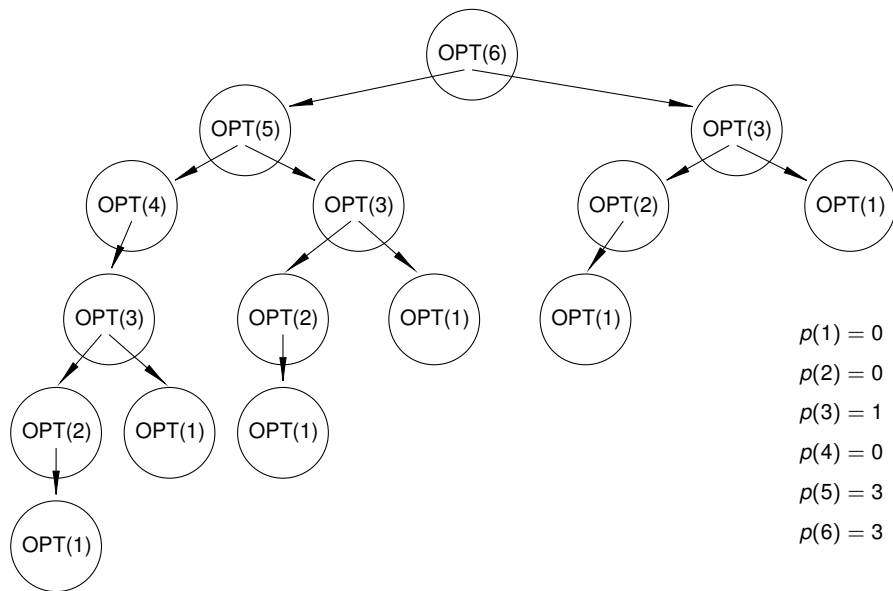
For $n = 6$ and $p(6) = 3$, we can

- 1 either not accept request 6 and move to request 5,
- 2 or accept request 6 and then move to request $p(6)$.

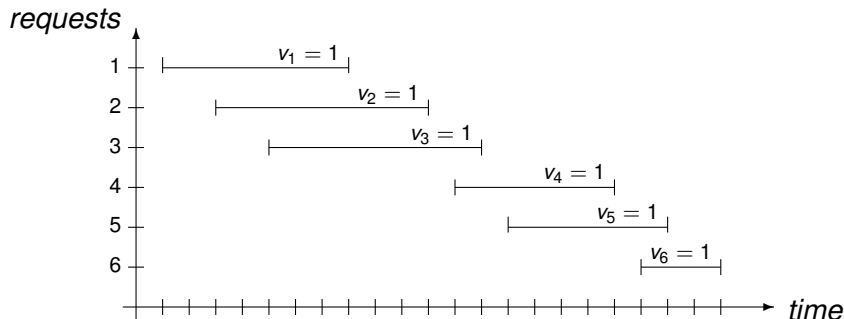
The first choice is shown at the left node,
the second choice is at the right node in the tree below:



the tree of subproblems



exponential growth



Exercise 1:

Consider the input shown above.

- 1 Run the algorithm `Compute-Opt` on the above input.
- 2 Show that the tree of subproblems is exponential in the input size.

Dynamic Programming

1 Introduction to Dynamic Programming

- weighted interval scheduling
- the design of a recursive solution
- **memoizing the recursion**
- longest common subsequence

2 Principles of Dynamic Programming

- another algorithm design technique
- memoization or iteration over subproblems
- the direct iterative algorithm
- a basic outline of dynamic programming

memoizing the recursion

The growth of the tree of subproblems is similar to the exponential growth of the recursive algorithm to compute the Fibonacci numbers

$$F(0) = 0, F(1) = 1, \quad \text{for } n > 1 : F(n) = F(n-1) + F(n-2).$$

We can make the recursion efficient by

- storing the results of previous calls to the function F in an array M .
- Prior to making the recursive call, M is consulted.

Euclid's algorithm computes the greatest common divisor in n steps when given $F(n+1)$ and $F(n+2)$ on input.

Donald E. Knuth. *The Art of Computer Programming. Volume 2. Seminumerical Algorithms*. Third Edition, Addison-Wesley, 1998.

the memoized version of the Fibonacci recursion

$M-F(n, M)$

Input: n , index to the n -th Fibonacci number,
 M , previously computed values of F .

if $M[n]$ is defined then

 return $M[n]$

else

 if $n = 0$ then

$M[n] := 0$

 else if $n = 1$ then

$M[n] := 1$

 else

$M[n] := M-F(n-1, M) + M-F(n-2, M)$

 return $M[n]$

the memoized version of Compute-Opt

M-Compute-Opt(j, v, p, M)

Input: j , the index of the current request,
 v , the array of values for each request,
 p , the function p for the requests,
 M , previously computed values of OPT.

if $M[j]$ is defined then

 return $M[j]$

else

 if $j = 0$ then

$M[j] := 0$

 else

$M[j] := \max(\text{M-Compute-Opt}(p(j), v, p, M) + v_j,$
 $\text{M-Compute-Opt}(j - 1, v, p, M))$

 return $M[j]$

cost of the memoized version

Theorem (5)

The running time of $M\text{-Compute-Opt}(n, v, p)$ is $O(n)$.

Proof. One single call to $M\text{-Compute-Opt}$ is $O(1)$.

Accessing the array M for the value $M[n]$ is $O(1)$.

The assignment to $M[n]$ is also $O(1)$. The other $O(1)$ operations are one addition $+v_n$ and the execution of $\max()$.

We have to show that the total number of calls to $M\text{-Compute-Opt}$ is $O(n)$.

The number of elements in the array M is $n + 1$.

Each recursive call to $M\text{-Compute-Opt}$ leads to two calls of $M\text{-Compute-Opt}$, but results in one new value in M that is defined.

Since M has $n + 1$ entries, the number of calls is $O(n)$.

Q.E.D.

computing a solution in addition to its value

We can recover the solution from the array M .

Find-Solution(j, v, p, M)

Input: j , the index of the current request,
 v , the array of values for each request,
 p , the function p for the requests,
 M , previously computed values of OPT.

if $j = 0$ then

 print nothing

else

 if $v_j + M[p(j)] \geq M[j - 1]$ then

 print j and do Find-Solution($p(j), v, p, M$)

 else

 Find-Solution($j - 1$)

Dynamic Programming

1 Introduction to Dynamic Programming

- weighted interval scheduling
- the design of a recursive solution
- memoizing the recursion
- longest common subsequence

2 Principles of Dynamic Programming

- another algorithm design technique
- memoization or iteration over subproblems
- the direct iterative algorithm
- a basic outline of dynamic programming

longest common subsequence

Given two sequences,
what is the length of the longest common subsequence?

A sequence $z_1, z_2, \dots, z_k$ is a subsequence of x
if there is a strictly increasing sequence of indices $i_1 < i_2 < \dots < i_k$
such that $x_{i_j} = z_j$, for $j = 1, 2, \dots, k$.

Example: b, c, d, b is a subsequence of a, b, c, b, d, a, b
for indices 2, 3, 5, 7.

For given sequences x and y , the longest common subsequence

- either ends at x_i and y_j ,
- or ends at x_i and y_{j-1} ,
- or ends at x_{i-1} and y_j ,

where i and j are valid indices in x and y .

a recursion

Let $c_{i,j}$ be the length of a common subsequence of $x_1\dots x_i$ and $y_1\dots y_j$, then

$$c_{i,j} = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ 1 + c_{i-1,j-1} & \text{if } i > 0, j > 0 \text{ and } x_i = y_j \\ \max(c_{i,j-1}, c_{i-1,j}) & \text{if } i > 0, j > 0 \text{ and } x_i \neq y_j \end{cases}$$

Exercise 2:

- 1 Given two sequences x and y of length m and n respectively, define an algorithm to compute $c_{i,j}$ for all i ranging from 0 to m and for all j ranging from 0 to n , using memoization.
- 2 What is the cost of your algorithm? Justify your answer.

evaluating Chebyshev polynomials

Exercise 3:

Chebyshev polynomials can be computed via the recursion:

$$T_0(x) = 1, \quad T_1(x) = x, \quad T_n(x) = 2xT_{n-1}(x) - T_{n-2}(x).$$

Consider a (complex) number z and a degree n .

- 1 What is the problem with the straightforward recursion applied to compute $T_n(z)$?
- 2 Define the application of memoization to compute $T_n(z)$.
Derive an upper bound on the running time to compute $T_n(z)$.
Justify your upper bound.

V. L. Goncharov: The Theory of Best Approximation of Functions.
Journal of Approximation Theory 106:2–57, 2000.
describes the 1854 memoir of Chebyshev on
Théorie des mécanismes connus sous le nom de parallélogrammes.

Dynamic Programming

1 Introduction to Dynamic Programming

- weighted interval scheduling
- the design of a recursive solution
- memoizing the recursion
- longest common subsequence

2 Principles of Dynamic Programming

- another algorithm design technique
- memoization or iteration over subproblems
- the direct iterative algorithm
- a basic outline of dynamic programming

CS 401/MCS 401 Lecture 12
Computer Algorithms I
Jan Verschelde, 15 July 2026

Dynamic Programming

1 Introduction to Dynamic Programming

- weighted interval scheduling
- the design of a recursive solution
- memoizing the recursion
- longest common subsequence

2 Principles of Dynamic Programming

- **another algorithm design technique**
- memoization or iteration over subproblems
- the direct iterative algorithm
- a basic outline of dynamic programming

another algorithm design technique

In earlier lectures we covered

- 1 greedy algorithms, which are often most natural to use first; and
- 2 divide and conquer, for faster running times of efficient algorithms.

Often, no natural greedy algorithm works.

With dynamic programming, we explore all subproblems, similar to an exhaustive search.

The original problem is decomposed into a series of subproblems and the solution is built from the solutions to subproblems.

Dynamic Programming

1 Introduction to Dynamic Programming

- weighted interval scheduling
- the design of a recursive solution
- memoizing the recursion
- longest common subsequence

2 Principles of Dynamic Programming

- another algorithm design technique
- **memoization or iteration over subproblems**
- the direct iterative algorithm
- a basic outline of dynamic programming

memoization or iteration over subproblems

Dynamic programming solves an optimization problem

- through an exploration of subproblems,
- building up solutions to larger and larger subproblems.

While the set of all possible solutions is exponentially large, not all possibilities are examined.

For the weighted interval scheduling problem, with memoization we turned an exponential-time recursive algorithm into a polynomial-time algorithm.

$$F(n) = F(n - 1) + F(n - 2), F(1) = 1, F(0) = 0$$

computed with memoization

$M-F(n, M)$

Input: n , index to the n -th Fibonacci number,
 M , previously computed values of F .

if $M[n]$ is defined then

return $M[n]$

else

if $n = 0$ then

$M[n] := 0$

else if $n = 1$ then

$M[n] := 1$

else

$M[n] := M-F(n - 1, M) + M-F(n - 2, M)$

return $M[n]$

towards an iterative version

Running the memoized version of the Fibonacci numbers ...

```
M-F(5, M = [])  
-> M-F(4, M = [])  
  -> M-F(3, M = [])  
    -> M-F(2, M = [])  
      -> M-F(1, M = []), base case M[1] := 1  
      -> M-F(0, M = [,1]), base case M[0] := 0  
      M[2] := 1+0, return calls with 1 and 0  
    -> M-F(1, M = [0,1,1])  
      M[3] := 1 + 1, return calls with 2 and 1  
    -> M-F(2, M = [0,1,1,2])  
      M[4] := 2 + 1, return calls with 3 and 2  
    -> M-F(3, M = [0,1,1,2,3])  
      M[5] := 3 + 2, return calls with 4 and 3
```

focus on the evolution of M

From the run on $M-F(5, M)$, focus on M :

$M = []$

$M = [, 1]$

$M = [0, 1, 1]$

$M = [0, 1, 1, 2]$

$M = [0, 1, 1, 2, 3]$

$M = [0, 1, 1, 2, 3, 5]$

The progress of M suggests the following algorithm:

$M[0] := 0$

$M[1] := 1$

for i from 2 to n do

$M[i] := M[i - 1] + M[i - 2]$

Dynamic Programming

1 Introduction to Dynamic Programming

- weighted interval scheduling
- the design of a recursive solution
- memoizing the recursion
- longest common subsequence

2 Principles of Dynamic Programming

- another algorithm design technique
- memoization or iteration over subproblems
- **the direct iterative algorithm**
- a basic outline of dynamic programming

the weighted interval scheduling problem

solved with memoization

M-Compute-Opt(j, v, p, M)

Input: j , the index of the current request,
 v , the array of values for each request,
 p , the function p for the requests,
 M , previously computed values of OPT.

if $M[j]$ is defined then
 return $M[j]$

else

 if $j = 0$ then

$M[j] := 0$

 else

$M[j] := \max(\text{M-Compute-Opt}(p(j), v, p, M) + v_j,$
 $\text{M-Compute-Opt}(j - 1, v, p, M))$

 return $M[j]$

the direct iterative algorithm

For the weighted interval scheduling problem,
we can formulate an iterative algorithm directly.

`Iterative-Compute-Opt( $v, p$ )`

Input: v , the array of values for each request,
 p , the function p for the requests.

```
 $M[0] := 0$   
for  $j = 1, 2, \dots, n$  do  
     $M[j] := \max(v_j + M[p(j)], M[j - 1])$ 
```

running Iterative-Compute-Opt

Iterative-Compute-Opt(v, p)

Input: v , the array of values for each request,
 p , the function p for the requests.

$M[0] := 0$

for $j = 1, 2, \dots, n$ do

$M[j] := \max(v_j + M[p(j)], M[j - 1])$

j	v_j	$p(j)$	$M[j]$
0			0
1	2	0	
2	4	0	
3	4	1	
4	7	0	
5	2	3	
6	1	3	

running Iterative-Compute-Opt

Iterative-Compute-Opt(v, p)

Input: v , the array of values for each request,
 p , the function p for the requests.

$M[0] := 0$

for $j = 1, 2, \dots, n$ do

$M[j] := \max(v_j + M[p(j)], M[j - 1])$

j	v_j	$p(j)$	$M[j]$
0			0
1	2	0	2
2	4	0	
3	4	1	
4	7	0	
5	2	3	
6	1	3	

running Iterative-Compute-Opt

Iterative-Compute-Opt(v, p)

Input: v , the array of values for each request,
 p , the function p for the requests.

$M[0] := 0$

for $j = 1, 2, \dots, n$ do

$M[j] := \max(v_j + M[p(j)], M[j - 1])$

j	v_j	$p(j)$	$M[j]$
0			0
1	2	0	2
2	4	0	4
3	4	1	
4	7	0	
5	2	3	
6	1	3	

running Iterative-Compute-Opt

Iterative-Compute-Opt(v, p)

Input: v , the array of values for each request,
 p , the function p for the requests.

$M[0] := 0$

for $j = 1, 2, \dots, n$ do

$M[j] := \max(v_j + M[p(j)], M[j - 1])$

j	v_j	$p(j)$	$M[j]$
0			0
1	2	0	2
2	4	0	4
3	4	1	6
4	7	0	
5	2	3	
6	1	3	

running Iterative-Compute-Opt

Iterative-Compute-Opt(v, p)

Input: v , the array of values for each request,
 p , the function p for the requests.

$M[0] := 0$

for $j = 1, 2, \dots, n$ do

$M[j] := \max(v_j + M[p(j)], M[j - 1])$

j	v_j	$p(j)$	$M[j]$
0			0
1	2	0	2
2	4	0	4
3	4	1	6
4	7	0	7
5	2	3	
6	1	3	

running Iterative-Compute-Opt

Iterative-Compute-Opt(v, p)

Input: v , the array of values for each request,
 p , the function p for the requests.

$M[0] := 0$

for $j = 1, 2, \dots, n$ do

$M[j] := \max(v_j + M[p(j)], M[j - 1])$

j	v_j	$p(j)$	$M[j]$
0			0
1	2	0	2
2	4	0	4
3	4	1	6
4	7	0	7
5	2	3	8
6	1	3	

running Iterative-Compute-Opt

Iterative-Compute-Opt(v, p)

Input: v , the array of values for each request,
 p , the function p for the requests.

$M[0] := 0$

for $j = 1, 2, \dots, n$ do

$M[j] := \max(v_j + M[p(j)], M[j - 1])$

j	v_j	$p(j)$	$M[j]$
0			0
1	2	0	2
2	4	0	4
3	4	1	6
4	7	0	7
5	2	3	8
6	1	3	8

proof of correctness

Obviously, for n requests,
the running time of `Iterative-Compute-Opt` is $O(n)$.

Exercise 4:

Prove that `Iterative-Compute-Opt` is correct.

number of ways to parenthesize a multiplication

Matrices A , B , C : A is 300-by-200, B is 200-by-100, C is 100-by-300. Although $(AB)C = A(BC)$, the number of scalar multiplications differs:

$(AB)C$:

- 1 AB requires 200 multiplications per element in AB , and AB has 300×100 elements: $300 \times 100 \times 200$ multiplications.
- 2 $(AB)C$ requires 100 multiplications per element in $(AB)C$, and $(AB)C$ has 300×300 elements: $300 \times 300 \times 100$ multiplications.

$A(BC)$:

- 1 BC requires 100 multiplications per element in BC , and BC has 200×300 elements: $200 \times 300 \times 100$ multiplications.
- 2 $A(BC)$ requires 200 multiplications per element in $A(BC)$, and $A(BC)$ has 300×300 elements: $300 \times 300 \times 200$ multiplications.

a recursion for the number of ways to parenthesize

Let $s(n)$ be the number of ways to put parentheses in a chain of multiplication of n matrices of compatible dimensions.

- If $n = 1$, then obviously $s(1) = 1$.
- If $n > 1$, then we can split the sequence in two halves:
 - 1 the first half is the first k matrices in the sequence,
 - 2 the second half contains the matrices $k + 1$ to n .for k running from 1 to $n - 1$.

Counting all the ways to parenthesize is thus done by the recursion

$$s(1) = 1, \quad \text{for } n > 1 : s(n) = \sum_{k=1}^{n-1} s(k)s(n-k).$$

apply memoization

Exercise 5:

The number $s(n)$ of ways to put parentheses in a chain of multiplication of n matrices of compatible dimensions is determined by the recursion:

$$s(1) = 1, \quad \text{for } n > 1 : s(n) = \sum_{k=1}^{n-1} s(k)s(n-k).$$

- 1 Define the application of memoization to compute $s(n)$.
- 2 Derive an upper bound on the running time to compute $s(n)$. Justify your upper bound.
- 3 Write a direct iterative algorithm to compute $s(n)$.

on the growth of $s(n)$

Exercise 6:

Apply the substitution method to

$$s(1) = 1, \quad \text{for } n > 1 : s(n) = \sum_{k=1}^{n-1} s(k)s(n-k)$$

to determine the growth of $s(n)$.

Does $s(n)$ grow as a polynomial in n ? Or exponential?

Dynamic Programming

1 Introduction to Dynamic Programming

- weighted interval scheduling
- the design of a recursive solution
- memoizing the recursion
- longest common subsequence

2 Principles of Dynamic Programming

- another algorithm design technique
- memoization or iteration over subproblems
- the direct iterative algorithm
- a basic outline of dynamic programming

a basic outline of dynamic programming

For the weighted interval scheduling, we have two efficient algorithms:

- 1 M-Compute-Opt is a memoized recursion, and
- 2 Iterative-Compute-Opt is a direct iterative algorithm.

Both algorithms are derived from the recurrence

$$\text{OPT}(j) = \max(v_j + \text{OPT}(p(j)), \text{OPT}(j - 1)).$$

The iterative building up of subproblems leads to algorithms that are often simpler to express.

guidelines for dynamic programming

For dynamic programming to apply, we need a collection of subproblems derived from the original problem that satisfies the properties:

- 1 There are a polynomial number of subproblems.
- 2 The solution to the original problem can be easily computed from the solutions to the subproblems.
- 3 There is a natural order from smallest to largest subproblem, jointly with an easy-to-compute recurrence.

There is a chicken-and-egg relationship between

- the decomposition into subproblems; and
- the recurrence linking the subproblems.